SpecBridge AI — Product Brief

Track: 1 — PRD-to-Engineering Gap Analyzer
Status: Phase 1 / pre-build hypotheses
Date: 16 October 2026

1. Problem Statement

Product requirements often reach engineering with implicit assumptions, undefined inputs, incomplete state transitions and missing failure behavior. Engineers then request clarification during estimation or development, increasing rework and delaying delivery. Existing generic AI reviews may over-flag harmless omissions, making their recommendations difficult to trust. SpecBridge AI identifies only evidence-backed implementation gaps before handoff; it is a decision-support tool, not a replacement for product and engineering review.

Problem hypothesis: pre-handoff gap reviews will reduce clarification cycles. We have not yet validated this hypothesis with real users.

2. User Persona

Primary persona: a Product Manager or Product Owner preparing feature requirements for an engineering team. Goal: provide implementable specifications with fewer back-and-forth questions. Context: works in a ticketing or document workflow, often under deadline pressure; technical details and constraints may be distributed across sections. Pain: important behavioral decisions are discovered late.

Secondary stakeholder: engineering lead, who needs explicit success, failure, input and boundary behaviors. Out of scope for MVP: direct Jira/ADO integrations, organization-specific compliance checks, and automatic rewriting of production PRDs.

3. Jobs To Be Done

WHEN I am preparing to hand a PRD to engineers, I WANT to identify materially unclear requirements, undefined inputs and missing decision logic with evidence and targeted questions, SO THAT the team can implement with fewer clarification loops and less avoidable rework.

Functional outcome: an actionable, traceable gap report. Emotional outcome: higher confidence before handoff. Success is not a high count of warnings; it is useful warnings with minimal noise.

4. Why AI? Why Not AI Alone?

Deterministic logic handles schema checks, required-field validation, length constraints, detectable conditional language, baseline completeness signals and output verification. Such checks are inexpensive, testable and consistent.

A language model adds value when deciding whether an apparent omission is material in the PRD context, resolving references across paragraphs, expressing trade-offs and crafting engineer-ready clarifying questions. AI does not invent requirements or claim that an unanswered question is definitely a defect.

Pipeline: structured form -> validation -> rule-based signals -> grounded prompt with original excerpts -> LLM API -> schema parsing -> evidence and confidence checks -> database -> structured results and admin dashboard. If the API fails, retain the rules-only signals but explicitly label analysis as incomplete; do not misrepresent them as full AI findings.

5. Feature List and Value vs Effort

MUST HAVE (high value, low-medium effort): title/context/PRD input; field and size validation; deterministic signals; one real server-side LLM call; classification into ambiguity, missing logic, undefined inputs; excerpt and explanation per finding; 2-3 questions; calibrated-language confidence indicator; saved analyses; history; protected admin dashboard; failure and retry states.

SHOULD HAVE (medium effort): severity, feedback buttons (helpful/not an issue), duplicate suppression, analysis latency/usage metadata, search history.

LATER (higher effort): file uploads, Jira/ADO, multi-user SSO, PDF export, organization templates, automatic redrafting. Scope control: no chat-only interface, no file ingestion in MVP.

6. Failure Mode Analysis

PRIMARY RISK - over-flagging: heuristics only nominate candidates; AI must cite the source excerpt, assess implementation impact, and suppress issues already resolved elsewhere. Limit findings by importance rather than fill a quota. Accept zero findings. Show user feedback buttons.

Other risks: hallucinated missing details -> require evidence quotation and no invented business policies; contradictions -> present both passages and allow unknown; low-confidence cases -> show needs-review label; missing/irrelevant input -> validate and show clear guidance; API timeouts/rate limits -> retry with bounds and truthful partial state; malformed outputs -> schema validation and safe error; sensitive PRDs -> use synthetic samples in public demos, keep secrets server-side, restrict admin access; long inputs -> cap size and disclose truncation rather than silently omit text.

Confidence: a transparent heuristic quality indicator initially, not an empirically calibrated probability. After evaluation data exists, study precision on hand-labeled test cases before using percentage-style confidence.

7. AI Trade-offs

LLM strengths: cross-section semantic interpretation and natural clarifying questions. Weaknesses: non-determinism, cost, delay, privacy risks and occasional unsupported

assertions. Mitigations: low-variance settings if supported, structured schema, quote verification, prompt-injection resilience (PRD treated as untrusted content), bounded tokens, fallback status, clear user review.

Do not overstate model accuracy. The initial prototype demonstrates a plausible workflow and tests; it does not prove causal improvement in engineering productivity.

8. Cost and Sustainability

Variable cost: tokens processed and generated per analysis, plus possible hosted AI integrations. Infrastructure cost: Replit hosting and persistent PostgreSQL depending on current plan. Control costs using maximum PRD length, output limits, model selection, rate limiting, capped usage during demo, and optional caching on exact document hash (with user permission). Monitor successful calls, errors, median latency, token usage if available, and estimated spend.

Example cost formula: (input tokens / 1,000,000 x input price) + (output tokens / 1,000,000 x output price). Prices depend on the provider/model and must be checked before activation. ChatGPT subscription billing is separate from OpenAI API billing. No costs or savings are guaranteed in advance.

9. Reflection and Validation Plan

Initial reflection: a product is useful only if recommendations help a team make decisions with less noise. Logic-first design provides testability; AI should add interpretation rather than obscure missing evidence. The MVP prioritizes trustworthy findings and explainability over breadth.

Research to complete: conduct at least 3 interviews or feedback sessions with PMs or engineers; evaluate a labeled set of at least 12 synthetic PRDs (clear, incomplete, mixed, conflicting); measure precision of flagged issues, false-positive count on clear PRDs, questions judged actionable, failure handling and repeatability. Targets proposed for evaluation, not achieved results: at least 80% of shown flags rated useful in a small pilot; at most one low-value flag on a clear control PRD; no invented source quotations. Record actual outcomes after testing and revise this reflection before submission.

10. Acceptance Criteria and MVP Boundaries

Given a valid PRD, when Analyze is clicked, the system runs rules first, invokes a real backend LLM API, validates output, displays categorized findings with source evidence and 2-3 questions, and stores the result. Given a clear PRD, the system may show no material flags. Given malformed/empty input, validation blocks the API. Given an API failure, the user sees an explicit failure or partial status and no fabricated AI result. Given a stored report, refreshing and reopening history retrieves it. Admin dashboard requires authorization and shows counts, error rates and feedback; no secret values or full sensitive PRDs exposed in summary analytics.

MVP excludes file upload, collaborative editing, SSO and direct work-management integrations.

11. GitHub Commit and Demo Plan

Commit 1 docs: product brief and JTBD; Commit 2 feat: structured input and validation; Commit 3 feat: deterministic rule engine; Commit 4 feat: real AI integration and schema checks; Commit 5 feat: database persistence; Commit 6 feat: dashboard and analytics. Add subsequent commits for tests and deployment. Keep commits meaningful and do not expose secrets.

Demo sequence: problem and JTBD; show input and validation; show rule candidates before LLM; demonstrate real AI results, evidence and confidence; test a clear PRD and an error; show history and dashboard; show repository commits. Assignment video instructions conflict (<5 minutes versus 5-8 minutes); request clarification, provisionally target 4-5 minutes.